

Python: exercises on loops with strings and lists

This notebook was generated in **student** mode.

Exercise 1: Shortest Words

Exercise Description

Shortest Words

Write a function `shortest_words` that:

- takes a list of words
- returns a list containing the shortest words in the list received as argument. The list will contain more than one word when there are multiple words with the same length.

Examples:

- `shortest_words([])` returns `[]`
- `shortest_words(['sheldon', 'cooper'])` returns `['cooper']`
- `shortest_words(['sheldon', 'cooper', 'howard'])` returns `['cooper', 'howard']`

NOTE: do not use any built-in function from Python to solve the exercise

Your solution

```
# Write your solution here
```

Test your solution

```
assert shortest_words([]) == []
assert sorted(shortest_words(['sheldon', 'cooper'])) == ['cooper']
assert sorted(shortest_words(['sheldon', 'cooper', 'howard'])) == ['cooper', 'howard']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: find how many equal numbers

Exercise Description

find how many equal numbers

Define a function `count_equals` that:

- takes as arguments four numbers
- returns:
 - the maximum number of equal numbers among the four

Example:

- `count_equals(1,2,3,4)` should return 0
- `count_equals(1,2,5,4)` should return 0
- `count_equals(1,2,2,2)` should return 3 because there are three 2 in the sequence
- `count_equals(1,1,1,2)` should return 3 because there are three 1 in the sequence

Your solution

```
# Write your solution here
```

Test your solution

```
assert count_equals(1,2,3,4) == 0
assert count_equals(1,5,3,4) == 0
assert count_equals(1,5,5,4) == 2
assert count_equals(1,5,1,4) == 2
assert count_equals(1,5,5,5) == 3
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Fibonacci's sequence

Exercise Description

Fibonacci's sequence

Define a function `fibonacci` that:

- takes as arguments:
 - an integer number n
- returns:
 - a list containing the first n numbers of the Fibonacci's sequence

NOTE: The Fibonacci sequence is a series of numbers where each number is the sum of the two previous ones, starting with 0 and 1. To calculate it, you begin with 0 and 1, then add these to get the next number. Continue this process to generate the sequence. It goes 0, 1, 1, 2, 3, 5, 8, and so on.

Your solution

```
# Write your solution here
```

Test your solution

```
assert fibonacci(1) == [0]
assert fibonacci(3) == [0,1,1]
assert fibonacci(7) == [0, 1, 1, 2, 3, 5, 8]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Collatz

Exercise Description

Collatz

Define a function `collatz` that:

- takes as argument an integer number `n`
- returns:
 - a list containing all the numbers generated by the Collatz conjecture (stopping when reaching 1)

NOTE: The Collatz Conjecture is a mathematical problem that starts with any positive integer. The process involves two steps: if the number is even, divide it by 2; if it's odd, multiply it by 3 and add 1. Repeat this process with the resulting number. The conjecture suggests that, no matter what number you start with, you'll eventually reach the number 1.

Your solution

```
# Write your solution here
```

Test your solution

```
assert collatz(12) == [6,3,10,5,16,8,4,2,1]
assert collatz(1) == []
assert collatz(2) == [1]
assert collatz(4) == [2,1]
assert collatz(19) == [58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16,
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Find missing number in a sequence

Exercise Description

Find missing number in a sequence

Define a function `find_missing` that:

- Takes a list of $n-1$ integers, which represents a sequence of numbers from 1 to n , but one number is missing.
- Returns the missing number.

NOTE: Suppose that there is always only one number missing

Example:

- `find_missing([1, 2, 4, 5])` should return 3.
- `find_missing([2, 3, 4, 6, 1])` should return 5.

Your solution

```
# Write your solution here
```

Test your solution

```
assert find_missing([1, 2, 4, 5]) == 3
assert find_missing([2, 3, 4, 6, 1]) == 5
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Prefixes of a String

Exercise Description

1. Define the string `s` equal to `The Big Bang Theory`
2. Create the empty list `p`
3. Using a loop, add all prefixes of `s` to `p`
4. Print `p`

Your solution

```
# Write your solution here
```

Test your solution

```
assert len(p) == 19
```

Test your solution

```
assert p[0] == "T"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Check Palindrome

Exercise Description

Check Palindrome

Define a function `is_palindrome` that:

- Takes a string as input.
- Returns True if the string is a palindrome, and False otherwise.

NOTE: A palindrome is a word, phrase, number, or other sequence of characters that reads the same forward and backward (**ignoring spaces, punctuation, and capitalization**).

Example:

- `is_palindrome("racecar")` should return True.
- `is_palindrome("hello")` should return False.
- `is_palindrome("A man, a plan, a canal, Panama")` should return True.

Your solution

```
# Write your solution here
```

Test your solution

```
assert is_palindrome("racecar")
assert not is_palindrome("hello")
assert is_palindrome("A man, a plan, a canal, Panama")
assert is_palindrome("121")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Check Postfixes of a String

Exercise Description

1. Define the string `s` equal to The Big Bang Theory
2. Define the list `p` equal to `["y", "ry", "ery", ""]`
3. Remove from `p` any string that is not a postfix of `s`
4. Print `p`

Your solution

```
# Write your solution here
```

Test your solution

```
assert p == ["y", "ry", ""]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Prime Numbers

Exercise Description

Define a function `is_prime` that:

- takes as arguments a list `L` of positive integers
- returns:
 - if the list is not empty: a new list where the *i*-th element is a boolean asserting whether the *i*-th element from `L` is a prime number
 - otherwise: `[]`

Your solution

```
# Write your solution here
```

Test your solution

```
assert is_prime([]) == []
```

Test your solution

```
assert is_prime([3, 4, 9, 11]) == [True, False, False, True]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Word Frequency

Exercise Description

Define a function `count_freq` that:

- takes as arguments:
 - a string `s`
 - a list `L` of words
- returns:
 - if the list is not empty: a new list where the *i*-th element is the number of occurrences in `s` of the *i*-th word from `L`
 - otherwise: `[]`

Your solution

```
# Write your solution here
```

Test your solution

```
assert count_freq("test", []) == []
```

